Splicing Should Respect Access Control

Document #:
P3473R0

Date:
2024-10-16

Audience:
EWG, WG21

Reply-to:
Steve Downey <sdowney@gmail.com>

Source:
https://github.com/steve-downey/wg21org

reflection-access-control.org

tags/P3473R0-0-gd110c7b

Table of Contents

1 P2966 Splicing does not check access by design

2 Existing Examples of Access Violation

3 There are use cases for writing to private members

4 Read access is not Safe

5 Private names become part of API

6 Allowing unchecked access is a fundamental change to C++

7 References

Abstract: P2996 (Revzin et al. 2024) ignores access control for member access splice. Member access control should always be respected.

1. P2966 Splicing does not check access by design

```
#include <experimental/meta>
#include <iostream>
#include <utility>

class S {
    int priv;
public:
    S() : priv(0) {}
};

consteval auto member_named(std::string_view name) {
  for (std::meta::info field : nonstatic_data_members_of(^S)) {
    if (has_identifier(field) && identifier_of(field) == name)
      return field;
  }
  std::unreachable();
}

int main() {
  S s;
  s.[:member_named("priv"):] = 42;
  return s.[:member_named("priv"):];
}
```

https://godbolt.org/z/6b1hf8631

This example is just slightly modified from an example in 3.3 of P2966. https://godbolt.org/z/MEPb78ece

> Note that a “member access splice” like `s.[:member_number(1):]` is a more direct member access mechanism than the traditional syntax. It doesn’t involve member name lookup, access checking, or — if the spliced reflection value represents a member function — overload resolution.

2. Existing Examples of Access Violation

It has been argued that C++ has existing mechanisms for access control avoidance. One in particular was presented to me.

```
class B {
    int priv;
  public:
    int get() {return priv;}
 };

int B::* get_private();

template <int B::* M>
struct Robber {
   friend int B::* get_private() { return M; };
};

template struct Robber<&B::priv>;

int main() {
    B b;
    b.*get_private() = 42;
    return b.get();
}
```

https://godbolt.org/z/eYvb97G91

Simplifying and making the mechanism less general, but possibly somewhat clearer:

```
class B { int priv;};

using tag = int B::*;
tag get_private();

template <tag M>
struct Robber {
   friend tag get_private() { return M; };
};

template struct Robber<&B::priv>;

int main() {
    B b;
    b.*get_private() = 42;
    return b.*get_private();
}
```

https://godbolt.org/z/1jqGMs1cr

Access to the name `B::priv` to take its address is not checked when creating the explicit instantiation of `Robber`. This includes the instantiation of the friend function defined in the body of the template, `get_private`, which has access to the template parameter object `M`, which holds the value of the member pointer to `B::priv`. Access control would normally apply to naming the type `Robber<Getter, &B::priv>`, but the friend function can be named without reference to the type that provided the definition. Access control is thereby skirted. The `Getter` type acts as a selection mechanism for overloads of `get_private`, so that no access check is violated at the call site. The `Robber` type would not even need to be visible in the same TU.

I believe this is a defect, even if there is no clear point at which to fix the problem. I do not believe it was a design choice to allow general avoidance of access control.

3. There are use cases for writing to private members

I believe these are not use cases that C++ should support. Deserialization is one that comes to mind, where writing to private data is imperative. Serialization/Deserialization is an important use case for reflection. It should be limited to code instantiated in contexts that have access to the members otherwise, without reflection, through member functions of a type, or friends that the type declares. While writing a deserializer for a type that doesn't support it now would be wonderful, the mechanism of writing to elements without checking access control is too blunt a tool, and too prone to casual misuse.

There is no way to restrict a general facility for only good and approved purposes. Language mechanisms are neutral. The tools will be misused and useful mechanisms will have to be banned because of the potential for misuse. The programmer cannot be trusted.

4. Read access is not Safe

Even read access is not generally safe in a multithreaded environment, which is becoming more and more common. Reads from containers are unsafe, and usually undefined, in the face of any write operation. Reads from non-atomic data is also unsafe if the data is being written. Other languages mitigate this in various ways. Rust's type system makes read and write access safe. Java has monitors for classes and objects idiomatically used to control multithreaded access. C++ has no such conventions. This sort of bug is common today in ostream operators, of course, and in formatters. Making a formatter safe requires understanding of the type, and from the outside it will not be clear to a library what the required techniques are.

5. Private names become part of API

The names of private data become part of the API of an object as changing them becomes a break in client code. Even without an ABI implication. Even if the client code can be changed, it increases the cost of any refactoring.

Hyrum's Law gets a new tool to couple dependencies. That it is out of contract behavior, and that the implementer is entitled to break the code of the client, is of little use in practice.

This is, admittedly, an exercise in line drawing. Access control is checked last, after name lookup, so private names can affect compilation today. However, someone checking for the existence of the name `lock` without using it in any way is not the same level of risk as someone looking for it and locking it from the outside.

6. Allowing unchecked access is a fundamental change to C++

We have so far resisted the temptation to deliberately provide tools to ignore access control.

We should continue to do so.

7. References

Revzin, Barry, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2024. “P2996R5: Reflection for c++26.” https://wg21.link/p2996r5; WG21.

Exported: 2024-10-16 15:49:35